Observabilidade de banco de dados

Seminário de Observabilidade em Sistemas Distribuídos — Tema 13

Relatório final | 23 de outubro de 2026

1. O problema

Uma API pode ficar lenta mesmo quando continua respondendo sem erros. Em um sistema com vários serviços, é difícil saber onde o tempo está sendo gasto: na comunicação entre eles, no código ou no banco de dados. Olhar apenas o tempo total da resposta não mostra a causa.

Neste projeto, criamos um caso chamado **N+1**. A aplicação busca uma lista de pedidos e depois faz outra consulta ao banco para descobrir o cliente de cada pedido. Para mostrar 100 pedidos, ela executa 101 consultas. Cada consulta pode ser rápida sozinha, mas a repetição aumenta o tempo total. A versão melhorada usa **JOIN**, recurso do SQL que reúne pedidos e clientes em uma consulta. As duas versões devolvem os mesmos pedidos, na mesma ordem.

2. A solução construída

O ambiente roda localmente com Docker Compose. A pedidos-api recebe a chamada do usuário e conversa por HTTP com a dados-api. Só a dados-api acessa o PostgreSQL. O banco começa, por padrão, com 1.000 clientes e 10.000 pedidos gerados de forma repetível.

Caminho da requisição: usuário → pedidos-api → dados-api → PostgreSQL.

Usamos dois tipos de informação para investigar o problema:

- **Traces** mostram o caminho de uma requisição. Cada etapa aparece como um span. OpenTelemetry liga as chamadas das duas APIs e as consultas SQL no mesmo trace; Jaeger mostra essa sequência.
- **Métricas** são números acompanhados ao longo do tempo. postgres_exporter coleta dados do banco, Prometheus os guarda e Grafana os apresenta. A extensão pg_stat_statements registra quantas vezes cada tipo de consulta foi executado e seu tempo acumulado.

Os serviços também registram erros em logs com códigos. Eles ajudam a reconhecer uma falha, mas não substituem o trace para seguir uma requisição nem os contadores para medir a repetição das consultas.

O trace de uma busca com 100 pedidos permite ver as 101 consultas da versão N+1 e a consulta única da versão com JOIN. Para conferir a quantidade exata, o experimento compara os contadores do banco antes e depois de cada execução. Assim, a causa da lentidão aparece nos dados, e não apenas em uma suspeita.

3. Como o teste foi feito

O teste comparou as duas rotas com os mesmos dados e limit=100. Foram três rodadas com uma chamada por vez e três rodadas com até cinco chamadas ao mesmo tempo. Cada execução teve 10 segundos de preparação e 30 segundos de medição. A ordem das rotas foi alternada para reduzir o efeito de uma sempre rodar primeiro. As respostas completas foram conferidas; uma resposta diferente seria tratada como erro.

Os valores abaixo vieram da execução local de **19/10/2026**. Cada número de tempo é a mediana dos resultados de três rodadas. O **p95** indica um tempo que pelo menos 95% das respostas válidas de uma rodada não ultrapassaram.

Chamadas simultâneas	Versão	Mediana das médias HTTP	Mediana dos p95
1	N+1	95,42 ms	167,45 ms
1	JOIN	11,46 ms	16,76 ms
5	N+1	203,84 ms	346,95 ms
5	JOIN	21,10 ms	38,18 ms

Nas 12 medições, houve **31.272 respostas válidas e nenhum erro**. Para cada resposta da rota N+1, o banco registrou 101 consultas da aplicação; para cada resposta com JOIN, registrou uma. O JOIN teve menor tempo de resposta nos dois níveis de concorrência. Isso apoia a explicação: fazer muitas consultas para montar a mesma resposta trouxe custo para esta aplicação.

4. Falha controlada na demonstração

O roteiro da apresentação também prevê parar o PostgreSQL **local** sem apagar seu volume. A API continua viva, mas deixa de estar pronta para buscar pedidos. O painel permite observar a perda de acesso ao banco, enquanto as rotas respondem com erro de dependência. Depois, o banco é iniciado novamente e a prontidão é verificada. Essa sequência ajuda a mostrar a diferença entre detectar uma falha e descobrir sua causa.

Essa falha do banco é um cenário separado do teste N+1. Não foi usada para produzir os números de latência da tabela.

5. Limitações e escolhas

Os resultados foram medidos em uma máquina local, com observabilidade ligada. Eles não garantem o mesmo ganho em outro computador ou sob outra carga. A distribuição dos dados é simples e não representa todos os usos reais. O teste não mediu a capacidade máxima do sistema nem separou o custo causado pela instrumentação.

Registrar todos os traces facilita a aula, mas consome recursos e pode gerar muitos dados. Jaeger e Prometheus guardam histórico temporário neste ambiente; reiniciar ou recriar componentes pode apagar parte dele. As métricas do Grafana são coletadas a cada cinco segundos e servem para acompanhar tendências. A contagem exata de consultas veio dos registros do PostgreSQL.

O pool de conexões, isto é, o limite de acessos simultâneos ao banco, foi configurado com até dez conexões. O projeto não realizou um teste próprio de esgotamento do pool. Também não criou cenários de espera por locks — quando uma operação bloqueia outra — nem de falta de índice. Existe um índice para a relação entre pedidos e clientes, mas não foi medida uma versão sem ele. Esses pontos fazem parte dos desafios do tema e ficam como limites claros desta demonstração.

Para este caso, o JOIN foi uma correção simples e verificável. Outra opção seria buscar vários clientes em uma segunda consulta e combinar os dados na aplicação. Isso poderia reduzir o número de consultas, mas exigiria mais código. Não implementamos essa alternativa porque uma consulta com JOIN resolveu o problema demonstrado.

6. Conclusão

A evidência principal para a lentidão da rota N+1 foi repetir consultas para cada pedido. Os traces mostraram essa repetição; os contadores do banco confirmaram a quantidade; e o teste de carga mostrou menor latência com JOIN nas condições medidas. O principal aprendizado é investigar o caminho completo de uma requisição antes de decidir como corrigir a lentidão.

Referências

1. PostgreSQL. pg_stat_statements, documentação da versão 18. https://www.postgresql.org/docs/18/pgstatstatements.html
2. OpenTelemetry. Traces. https://opentelemetry.io/docs/concepts/signals/traces/
3. Prometheus. Overview. https://prometheus.io/docs/introduction/overview/
4. Projeto, docs/experimento-carga.md: método, resultados e limites da medição de 19/10/2026.

Declaração de uso de IA

Usamos IA generativa (Codex, da OpenAI) como apoio para organizar e revisar a redação deste relatório. Os números apresentados foram retirados dos registros do experimento e da documentação do projeto; a IA não realizou uma nova medição. A equipe é responsável por conferir o texto e explicar o sistema e seus resultados na apresentação.